📝 DAV Lab Assignment 3: Seaborn Visualizations

Subject: Data Analysis & Visualization (DAV) | **Topic:** Categorical vs Numerical Distributions & Correlation | **Tool:** Python (Pandas, Seaborn, Matplotlib)

> 📌 **Lab Objective & Dataset Transition:**
> Previously, we completed this lab on dummy data (`earthquake.csv`) plotting *Alert Level vs Magnitude*.
> Now, we implement the ***exact same methodology*** on our real-world dataset: **`MoviesOnStreamingPlatforms.csv`**!

🔍 Mapping Old Variables ➔ New Dataset

- **Categorical Feature:** `Age` (*Rating:* `all, 7+, 13+, 16+, 18+`) *[Replaces 'alert']*
- **Numerical Feature:** `Rotten Tomatoes` *score [Replaces 'magnitude']*
- **Multiple Numerical Features:** `['Year', 'Rotten Tomatoes', 'Netflix', 'Hulu', 'Prime Video', 'Disney+']` *for Correlation Heatmap.*

⚡ Step 0: Setup, Libraries & Essential Data Cleaning

In the CSV, Rotten Tomatoes is saved as text like `"98/100"`*. We must strip the* `"/100"` *and convert it into a numeric* `float` *so Seaborn can compute quartiles!*

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load your dataset
df = pd.read_csv("MoviesOnStreamingPlatforms.csv")

# 2. Clean Rotten Tomatoes column: "98/100" → 98.0
df['Rotten Tomatoes'] = df['Rotten Tomatoes'].astype(str).str.replace('/100', '')
df['Rotten Tomatoes'] = pd.to_numeric(df['Rotten Tomatoes'], errors='coerce')

# 3. Drop missing values in Age and Rotten Tomatoes
plot_df = df.dropna(subset=['Age', 'Rotten Tomatoes']).copy()
plot_df = plot_df[plot_df['Age'].str.strip() ≠ '']

# 4. Set explicit categorical order
age_order = ['all', '7+', '13+', '16+', '18+']
```

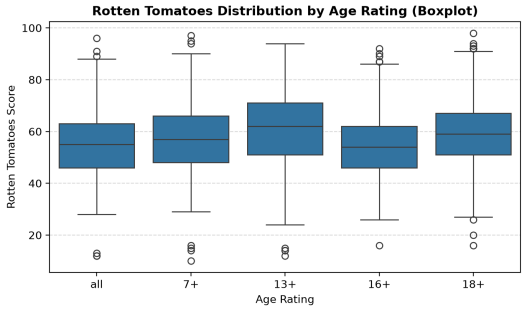
> 💡 **Why define `age_order`?** *By default, Seaborn sorts categories alphabetically (13+ before 7+). Defining* `order=age_order` *ensures chronological maturity order: all ➔ 7+*

➔ 13+ ➔ 16+ ➔ 18+!

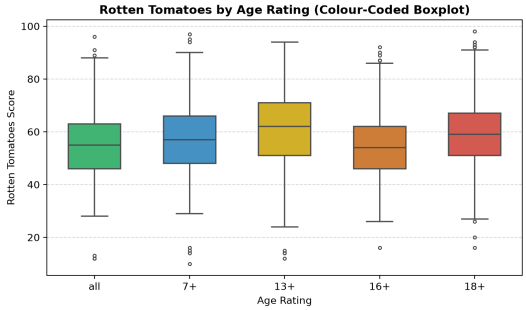
📦 Step 1 & 1b: Boxplot & Colour-Coded Boxplot

What is a Boxplot? It shows the *5-number summary* of data: **Minimum, Q1 (25%), Median (Q2 50%), Q3 (75%), and Maximum**. Outliers are shown as individual dots beyond the whiskers.

Standard Boxplot



Color-Coded Boxplot



💻 Implementation Code:

```
# Custom Color Palette Mapping
palette = {
    'all': '#2ecc71',   # Green
    '7+':  '#3498db',   # Blue
    '13+': '#f1c40f',   # Yellow
    '16+': '#e67e22',   # Orange
    '18+': '#e74c3c'    # Red
}

plt.figure(figsize=(10, 6))
sns.boxplot(
    data=plot_df,
    x='Age',
    y='Rotten Tomatoes',
    order=age_order,
    palette=palette,
    width=0.6,
    linewidth=1.2,
    fliersize=3
)
plt.title('Rotten Tomatoes Distribution by Age Rating (Colour-Coded Boxplot)')
plt.xlabel('Age Rating')
plt.ylabel('Rotten Tomatoes Score')
plt.tight_layout()
plt.show()
```

🧠 **Anatomy & Observations for Viva:**

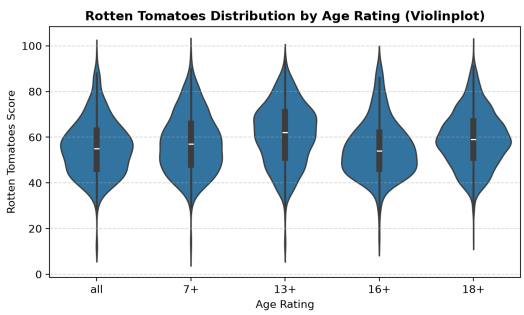
- **Median Line:** The horizontal bar inside each box. The median Rotten Tomatoes score across all age ratings hovers between ~45 to 55.

- **Interquartile Range (IQR = Q3 - Q1):** The height of the box contains the middle 50% of movies.
- **Fliers/Dots:** Extreme critically acclaimed movies (scores > 90) or critically panned movies appear as outliers!

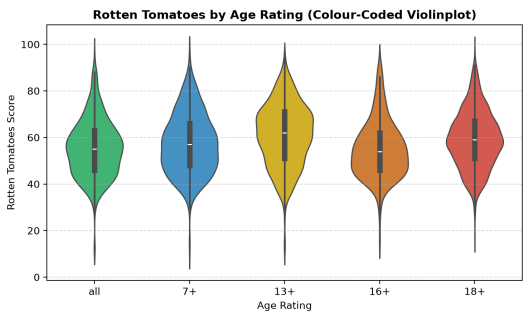
🎻 Step 2 & 2b: Violinplot & Colour-Coded Violinplot

Why use a Violin Plot over a Boxplot? A boxplot only shows statistical summaries, but a **Violin Plot** combines a mini boxplot with a *Kernel Density Estimate (KDE)* on both sides! The wider the violin at any point, the higher the concentration of movies with that score.

Standard Violinplot



Color-Coded Violinplot



💻 Implementation Code:

```
plt.figure(figsize=(10, 6))
sns.violinplot(
    data=plot_df,
    x='Age',
    y='Rotten Tomatoes',
    order=age_order,
    palette=palette,
    width=0.6,
    linewidth=1.2
)
plt.title('Rotten Tomatoes by Age Rating (Colour-Coded Violinplot)')
plt.xlabel('Age Rating')
plt.ylabel('Rotten Tomatoes Score')
plt.tight_layout()
plt.show()
```

🔍 **Graph Insights:** Notice the "bulge" in the middle of each violin around 40-55. This confirms that the majority of streaming movies receive average to mediocre scores, with long thin tails stretching towards 90-100.

📊 Step 3: Histplot with KDE Curve

In Step 3, we analyze the univariate continuous distribution of our target variable (`Rotten Tomatoes`*) across the whole catalog.*

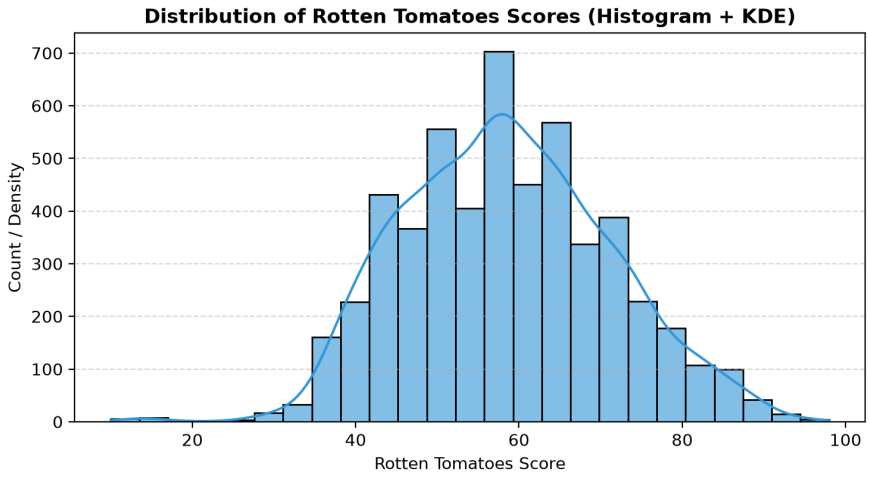


Figure: Histogram (bins=25) + Kernel Density Estimation (KDE) curve

```
plt.figure(figsize=(8, 5))
sns.histplot(
    data=plot_df,
    x='Rotten Tomatoes',
    kde=True,
    bins=25,
    color='#3498db'
)
plt.title('Distribution of Rotten Tomatoes Scores (Histogram + KDE)')
plt.xlabel('Rotten Tomatoes Score')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

🎯 **What `kde=True` does:** *The smooth curved line drawn over the bars is the Kernel Density Estimate. It transforms discrete bin counts into a smooth continuous probability density function.*

🔥 Step 4: Correlation Matrix & HeatMap

A heatmap visualizes linear correlations between pairs of numerical variables using the Pearson correlation formula (-1.00 = negative, 0.00 = no correlation, +1.00 = positive).

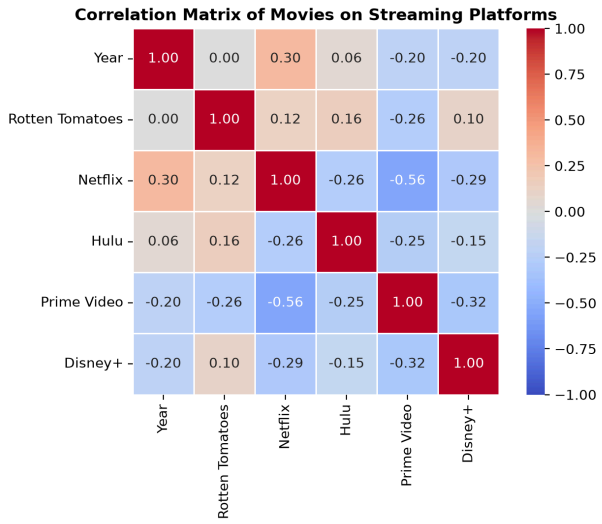


Figure: Correlation Heatmap with annot=True and coolwarm diverging colormap

```
nums_cols = ['Year', 'Rotten Tomatoes', 'Netflix', 'Hulu', 'Prime Video', 'Disney
corr = plot_df[nums_cols].corr()

plt.figure(figsize=(7, 5.5))
sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    fmt=".2f",
    square=True
)
plt.title('Correlation Matrix (Year, Rotten Tomatoes, Streaming Platforms)')
plt.tight_layout()
plt.show()
```

🔑 ***Parameter Explanations (Exam Favourite!):***

- `annot=True`: *Writes numerical values directly inside each square cell.*
- `cmap='coolwarm'`: *Blue for negative correlation, White for zero, Red for positive correlation.*
- `vmin=-1, vmax=1`: *Anchors the color scale strictly between -1 and +1.*
- `fmt=".2f"`: *Formats decimal values to 2 decimal places.*

🎓 Quick Revision & Lab Viva Cheat Sheet

❓ Q1: Why did we use Seaborn instead of pure Matplotlib?
Answer: Seaborn is built directly on top of Matplotlib and integrates tightly with Pandas DataFrames. It handles statistical aggregation, confidence intervals, automatic labels, and aesthetic coloring in 1 line of code.

❓ Q2: What is the difference between a Boxplot and a Violinplot?
Answer: A boxplot reveals the 5-number summary (quartiles and outliers) but assumes unimodal behavior. A violin plot shows the exact probability density shape at different values (can reveal bimodal/multimodal distributions).

❓ Q3: Why was data cleaning necessary for 'Rotten Tomatoes'?
Answer: The raw column contained strings with `"/100"` (e.g. `"98/100"`). Statistical tools require numeric floating-point values to compute means, medians, and correlations.

❓ Q4: What does the correlation matrix tell us about platforms?
Answer: The platform columns (Netflix, Hulu, Prime Video, Disney+) are binary indicators (0 or 1). Slight negative correlations between platforms indicate that most movies are licensed exclusively to a single platform rather than shared across all.

📋 Summary Checklist for Submission:

- ✅ Cleaned `Rotten Tomatoes` text column to float.
- ✅ Filtered missing/empty `Age` values.
- ✅ Generated standard and color-coded Boxplots using `order=age_order`.
- ✅ Generated standard and color-coded Violinplots.
- ✅ Generated Histogram with KDE overlay curve.
- ✅ Computed correlation matrix and rendered annotated `coolwarm` Heatmap.

🌟 Complete Lab Assignment 3 successfully ported to MoviesOnStreamingPlatforms.csv!